

DAY 10

Smart Contract Security I: Common Vulnerabilities & Secure Design

Reentrancy, access control, external calls, ordering, oracles, signatures and exploit-style testing

60-MINUTE LECTURE Beginner -> security-aware Solidity developer. Today you learn to think like a reviewer: identify assets, trust boundaries, privileged functions and external interactions before trusting the code.

Smart Contract Security = Protect Trust Boundaries



CAREER OUTCOME After this lecture, you should be able to recognize several high-impact vulnerability classes, reproduce a bug safely in a local test, explain the root cause to a client, and propose a defensible remediation.

1. Learning Objectives & 60-Minute Teaching Plan

- Explain why smart contract security is primarily about protecting assets, permissions and invariants against untrusted inputs and external behavior.
- Recognize reentrancy, missing access control, unchecked low-level calls, tx.origin authorization, signature replay, oracle manipulation and denial-of-service patterns.
- Apply checks-effects-interactions (CEI), least privilege, input validation, robust oracle design, nonce/domain-bound signatures and emergency controls.
- Write exploit-style Foundry tests that demonstrate a vulnerability before applying a fix.
- Communicate security findings in the language a freelance client can act on: impact, evidence, remediation and residual risk.

Minutes	Topic	Teaching goal
0-5	Security mindset + threat model	Assets, actors, trust boundaries, invariants
5-17	Reentrancy	Root cause, CEI, guard, exploit test
17-27	Access control + external calls	Permissions, roles, low-level call handling
27-37	Front-running / MEV + oracle risk	Transaction ordering and manipulated inputs
37-45	tx.origin + signature replay	Authentication and authorization context
45-52	DoS, gas, validation, misc. pitfalls	Availability and defensive coding
52-58	Hands-on security review lab	VulnerableVault -> SecureVault
58-60	Quiz + assignment	Retention + portfolio task

INSTRUCTOR RULE Do not present security as a checklist that guarantees safety. The same code pattern can be safe or unsafe depending on state, call graph, privileges and assumptions.

2. Security Mindset: Assets, Actors, Invariants, Trust

A smart contract is public code running in an adversarial environment. Attackers do not need to use your UI; they can call public/external functions directly, compose contracts, borrow liquidity, reorder transactions where possible, and deliberately choose edge-case inputs.

Four questions before reading individual functions

1. What assets or rights are valuable? ETH, ERC-20 balances, NFT ownership, minting authority, upgrade authority, oracle influence, governance power.
2. Who are the actors? Untrusted user, owner, operator, role holder, token contract, DEX, oracle, bridge, callback receiver.
3. What invariants must always remain true? Example: total user claims must not exceed assets held; only MINTER_ROLE can mint; a signature must be usable once.
4. Where are the trust boundaries? Every external contract call, privileged key, signed message, oracle read and user-controlled parameter is a boundary.

Smart Contract Security = Protect Trust Boundaries

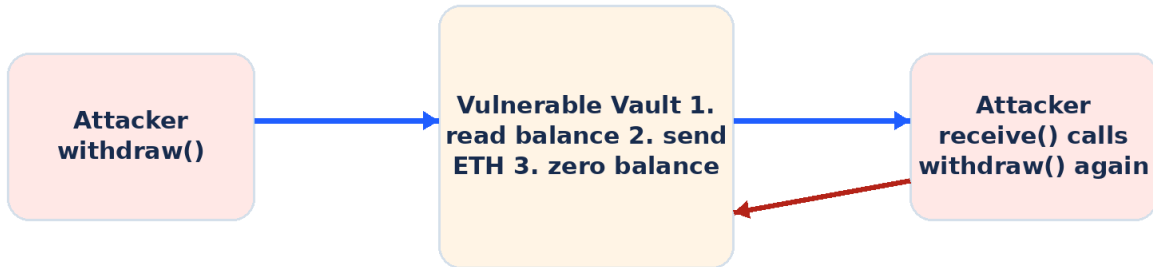


MENTAL MODEL A vulnerability is usually a broken assumption. The reviewer asks: "What assumption does this function make, and can an adversary make it false?"

3. Reentrancy: The Classic External-Call Hazard

Reentrancy happens when your contract makes an external call and the recipient executes code that calls back into your contract before the original execution has completed. If critical state still reflects the pre-withdrawal condition, the callback may reuse the same entitlement.

Reentrancy: External Call Before State Is Final



State still says attacker has funds -> repeated withdrawal path

Vulnerable ordering

```

function withdraw() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "no balance");

    // External interaction happens while state is still stale.
    (bool ok, ) = msg.sender.call{value: amount}("");
    require(ok, "send failed");

    balances[msg.sender] = 0;
}
  
```

ROOT CAUSE The dangerous fact is not “call is always bad.” The dangerous fact is that untrusted code runs before the contract has finalized the state that controls whether another withdrawal is allowed.

4. Reentrancy Defense: CEI + Guard + Narrow External Calls

Checks - Effects - Interactions (CEI)



Combine sound ordering with targeted guards and tests.

Safer state ordering

```

function withdraw() external {
    uint256 amount = balances[msg.sender];
    if (amount == 0) revert NothingToWithdraw();

    // EFFECT: consume entitlement before interaction.
    balances[msg.sender] = 0;

    // INTERACTION: untrusted code may run here.
    (bool ok, ) = payable(msg.sender).call{value: amount}("");
    if (!ok) revert TransferFailed();
}
  
```

Defense in depth with OpenZeppelin ReentrancyGuard

```

import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

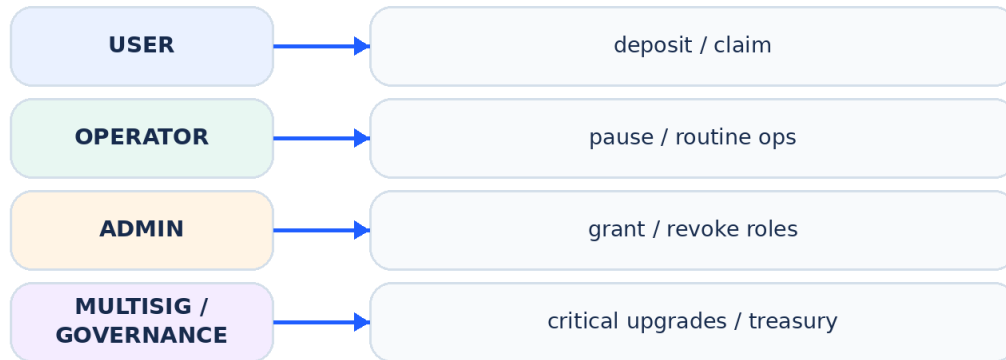
contract SecureVault is ReentrancyGuard {
    function withdraw() external nonReentrant {
        // checks -> effects -> interaction
    }
}
  
```

- Use CEI to make the state logically safe before external execution.
- Use a reentrancy guard on sensitive entry points when callbacks could create unsafe recursive paths.
- Remember cross-function and cross-contract reentrancy: protecting one function is not enough if another callable path can observe inconsistent state.
- Prefer pull-style withdrawals when a protocol would otherwise push payments to many recipients.

5. Access Control: The Most Expensive Missing Modifier

Access control answers “who may execute this sensitive action?” A perfect arithmetic implementation is still catastrophic if any caller can mint, pause, withdraw treasury assets, upgrade an implementation, change an oracle, or grant themselves a role.

Access Control: Who Is Allowed To Do What?



Vulnerable example

```
function setTreasury(address newTreasury) external {
    treasury = newTreasury; // Anyone can redirect funds.
}
```

Safer example

```
function setTreasury(address newTreasury) external onlyRole(ADMIN_ROLE) {
    if (newTreasury == address(0)) revert ZeroAddress();
    treasury = newTreasury;
}
```

- Use least privilege: separate routine operator rights from critical admin rights.
- Treat role-admin relationships as part of the security design; whoever can grant a role may effectively hold that role indirectly.
- For high-value systems, critical authority is often better held by multisig/governance/timelock arrangements than one hot EOA.
- Test unauthorized callers explicitly. A happy-path test cannot prove access control.

6. External Calls, ERC-20 Transfers & Return Values

Any external call can fail, reenter, consume unexpected gas, or return data you did not expect. Your contract should handle external behavior deliberately rather than assuming success.

Low-level call

```
(bool success, bytes memory data) = target.call(payload);
if (!success) {
    // Decide whether to bubble the revert or convert to a custom error.
    revert ExternalCallFailed();
}
```

ERC-20 integration

Token integrations can be tricky because deployed tokens are not perfectly uniform. OpenZeppelin SafeERC20 wraps common transfer/approve operations and handles tokens that do not return a conventional boolean value.

```
using SafeERC20 for IERC20;

function pay(address token, address to, uint256 amount) external {
    IERC20(token).safeTransfer(to, amount);
}
```

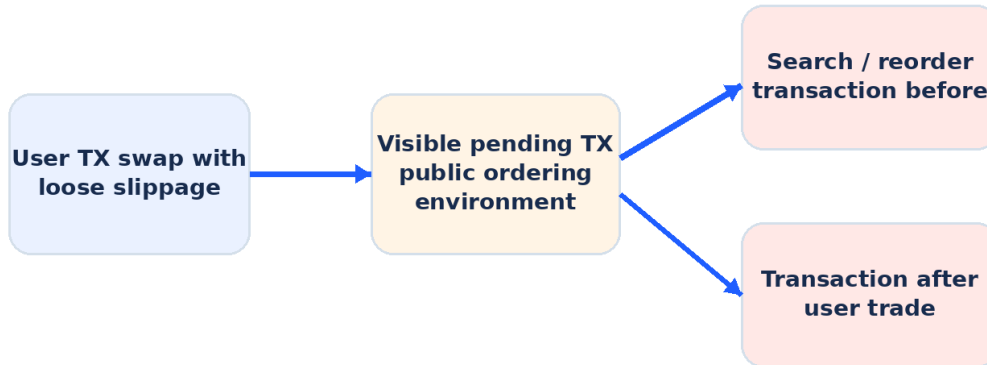
Risk	Bad assumption	Safer approach
Ignored return/failure	"transfer must have worked"	Use safe wrappers or inspect success/revert
Callback/reentrancy	"recipient is passive"	Assume recipient/target may execute code
Arbitrary target	"admin/user cannot abuse target"	Allowlist or tightly constrain call destinations
Approval misuse	"infinite allowance is harmless"	Minimize allowance scope and privilege

REVIEW QUESTION For every external call, ask: who controls the target, what state exists before the call, what can the target call back, and what happens if the call fails?

7. Front-Running, MEV & Transaction-Ordering Risk

Public blockchain transactions can become visible before final inclusion. Where block builders/validators/searchers can influence ordering, another transaction may execute before or after a user transaction and change the outcome.

Public Mempool Risk: Ordering Can Change Outcomes



Mitigate with slippage limits, deadlines, commit-reveal / private order flow where appropriate

Common client-level mitigations

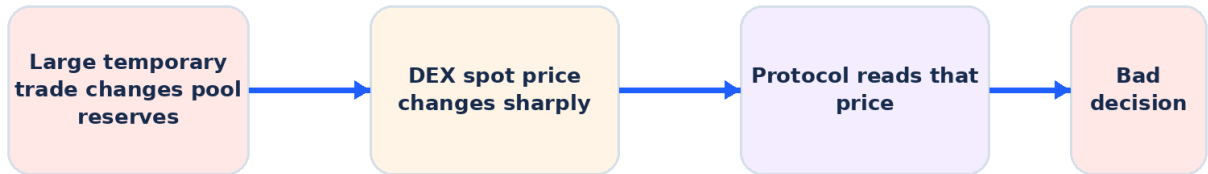
- Use explicit slippage limits rather than accepting any output amount.
- Use deadlines so a transaction cannot execute under stale market conditions indefinitely.
- Use commit-reveal schemes when revealing a secret choice early would enable copying/front-running.
- Consider private transaction/order-flow infrastructure for use cases where public mempool exposure is a core risk.
- Design auctions and liquidations with ordering behavior in mind; some MEV is an economic property, not a simple Solidity bug.

IMPORTANT Do not claim that a single modifier “prevents front-running.” Transaction ordering is a system/economic issue. Smart contract design can limit harmful consequences, but the right mitigation depends on the protocol.

8. Oracle Manipulation & Flash-Liquidity Context

A protocol becomes vulnerable when it treats a manipulable market observation as trustworthy truth. An attacker may temporarily move a thin DEX pool price, cause the protocol to read the distorted price, then profit from the protocol decision.

Oracle Manipulation: Spot Price Is Not Always Truth



Defenses: robust oracle design, freshness checks, TWAP/aggregation, bounds, c

Weak design	Why risky	More robust design idea
Single pool spot price	Can move sharply with temporary liquidity	Use robust oracle / aggregation / time-weighting where appropriate
No freshness check	Stale answer may be treated as current	Check update time / heartbeat assumptions
No bounds / sanity checks	Extreme values propagate directly	Validate ranges, decimals and circuit-breaker conditions
One dependency	Single point of data failure	Consider fallback / multiple sources if protocol requirements justify it

FLASH LOANS Flash loans do not create the underlying oracle bug. They can make temporary capital very cheap, which may make an existing manipulation path easier to exploit in a single transaction.

9. Authentication Mistake: Do Not Authorize With tx.origin

msg.sender is the immediate caller. tx.origin tracks the account that originated the transaction through the call chain. Using tx.origin for authorization can let an attacker contract trick the owner into initiating a transaction that reaches the vulnerable contract.

Bad pattern

```
function sweep(address payable to) external {
    require(tx.origin == owner, "not owner");
    to.call{value: address(this).balance}("");
}
```

Better principle

```
function sweep(address payable to) external onlyOwner {
    // onlyOwner checks the immediate authorized caller.
}
```

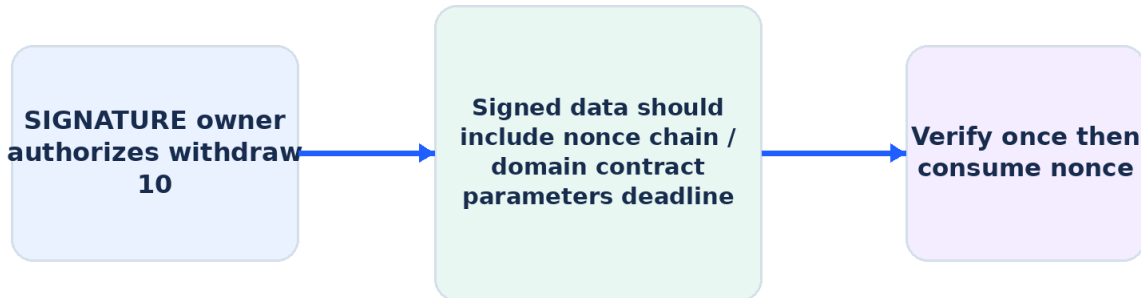
MODERN NOTE Security assumptions based on tx.origin are even less attractive as account models evolve. Authorization should use explicit caller/role/signature logic designed for the application.

- Use msg.sender or explicit role checks for normal contract-call authorization.
- For meta-transactions/account abstraction, use the framework-specific trusted-forwarder or signature model; do not improvise with tx.origin.
- Write tests where calls arrive through an intermediate contract.

10. Signature Replay: Valid Signature, Wrong Context

Off-chain signatures are powerful because users can authorize actions without directly submitting every transaction. But a signature that does not bind itself to a unique context may be reused.

Signature Replay Defense: Bind Authorization To Context



Authorization payload should normally bind

- The exact action and parameters being authorized.
- A nonce or unique identifier that is consumed after successful use.
- The verifying contract/domain so the signature cannot be replayed against another contract.
- The relevant chain/domain context where cross-chain replay would matter.
- A deadline/expiry when perpetual authorization is unsafe.

```

struct PermitLike {
    address user;
    address recipient;
    uint256 amount;
    uint256 nonce;
    uint256 deadline;
}
  
```

```
// Verify typed data -> verify nonce -> execute -> increment nonce.
```

EIP-712 MINDSET Typed structured data improves domain separation and human/tooling clarity, but replay protection still depends on correct nonce/domain/deadline logic.

11. Denial of Service, Unbounded Loops & Gas Griefing

A contract can be financially safe yet operationally unusable if one action can be made to consume too much gas or if one failing recipient blocks progress for everyone.

Unbounded loop risk

```
function payAll() external {
  for (uint256 i = 0; i < recipients.length; i++) {
    payable(recipients[i]).transfer(rewards[recipients[i]]);
  }
}
// As the list grows, this may become too expensive or one recipient may block execution.
```

Prefer pull-based claims for large/untrusted recipient sets

```
function claim() external nonReentrant {
  uint256 amount = claimable[msg.sender];
  claimable[msg.sender] = 0;
  (bool ok, ) = payable(msg.sender).call{value: amount}("");
  if (!ok) revert TransferFailed();
}
```

- Avoid loops whose cost can grow without a strict bound on a state-changing path.
- Separate accounting from payment so one receiver cannot necessarily block every receiver.
- Rate-limit or paginate administrative batch operations when necessary.
- Test worst-case sizes, not only lists with two elements.

12. Validation & Other High-Value Review Checks

Check	Example failure	Review question
Zero addresses	Admin/oracle/recipient set to address(0)	Should zero be valid here?
Bounds	Fee set to 10000% or duration to zero	What numeric ranges preserve invariants?
Decimals/units	USDC 6 decimals treated like 18	Are units explicit and tested?
Unchecked math	unchecked block wraps arithmetic	Is wraparound intended and proven safe?
block.timestamp	Used as precise randomness	Is miner/validator influence acceptable?
Contract ETH balance	Assumed to equal internal deposits	Can ETH arrive outside the normal deposit function?
Arbitrary delegatecall	Untrusted code executes in caller storage context	Who controls implementation/target?
Initializer / ownership	Implementation left uninitialized or owner wrong	Who owns/admins immediately after deployment?

SOLIDITY 0.8+ Arithmetic overflow/underflow is checked by default in normal Solidity arithmetic, but unchecked blocks, casts, assembly and logic errors still require review. "Solidity 0.8" is not a security certificate.

13. Hands-On Lab Part A: VulnerableVault

Use this contract only in a local test environment. The goal is to reproduce a reentrancy flaw and demonstrate why tests must encode attacker behavior.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract VulnerableVault {
    mapping(address => uint256) public balances;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() external {
        uint256 amount = balances[msg.sender];
        require(amount > 0, "nothing");

        // VULNERABILITY: interaction before effect.
        (bool ok, ) = payable(msg.sender).call{value: amount}("");
        require(ok, "send failed");

        balances[msg.sender] = 0;
    }
}
```

AUDIT NOTE The mapping is an entitlement ledger. The critical invariant is: a user must not withdraw more ETH than their recorded entitlement. The external call happens before the entitlement is consumed.

14. Hands-On Lab Part B: Local Attacker Contract

```
contract ReentrantAttacker {
    VulnerableVault public vault;
    uint256 public chunk;

    constructor(VulnerableVault _vault) {
        vault = _vault;
    }

    function attack() external payable {
        require(msg.value > 0, "seed needed");
        chunk = msg.value;
        vault.deposit{value: msg.value}();
        vault.withdraw();
    }

    receive() external payable {
        if (address(vault).balance >= chunk) {
            vault.withdraw();
        }
    }
}
```

- The receive callback is executed by the vault during the low-level ETH transfer.
- The callback calls withdraw again before balances[attacker] is reset.
- The local test funds the vault with test ETH and verifies that the attacker can withdraw more than the original deposit.
- This is evidence of the bug in a controlled environment, not a mainnet exploitation recipe.

15. Hands-On Lab Part C: Foundry Exploit-Style Test

```
function test_ReentrancyDrainsMoreThanDeposit() public {
    vm.deal(address(this), 20 ether);
    vm.deal(attackerEOA, 1 ether);

    // Seed vault with honest liquidity.
    vault.deposit{value: 10 ether}();

    vm.prank(attackerEOA);
    ReentrantAttacker a = new ReentrantAttacker(vault);

    vm.prank(attackerEOA);
    a.attack{value: 1 ether}();

    // Security assertion: attacker should never receive > entitlement.
    assertGt(address(a).balance, 1 ether);
}
```

TESTING PRINCIPLE A useful security test states the property the protocol intended to guarantee, then constructs a realistic adversarial sequence that attempts to falsify it.

Good test names communicate impact

- test_ReentrancyCannotWithdrawTwice()
- test_UnauthorizedCallerCannotSetOracle()
- test_SignatureCannotBeReused()
- test_StaleOracleAnswerReverts()
- test_BatchSizeCannotMakeFundsPermanentlyUnclaimable()

16. Hands-On Lab Part D: SecureVault

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {ReentrancyGuard} from
"@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract SecureVault is ReentrancyGuard {
    error NothingToWithdraw();
    error TransferFailed();

    mapping(address => uint256) public balances;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() external nonReentrant {
        uint256 amount = balances[msg.sender];
        if (amount == 0) revert NothingToWithdraw();

        // Effects first.
        balances[msg.sender] = 0;

        // Interaction last.
        (bool ok, ) = payable(msg.sender).call{value: amount}("");
        if (!ok) revert TransferFailed();
    }
}
```

- Regression test: reuse the attacker against SecureVault and assert that it cannot receive more than its legitimate amount.
- Add a normal-user test proving honest withdrawals still work after the fix.
- Security fixes should preserve intended functionality, not merely make the exploit test fail.

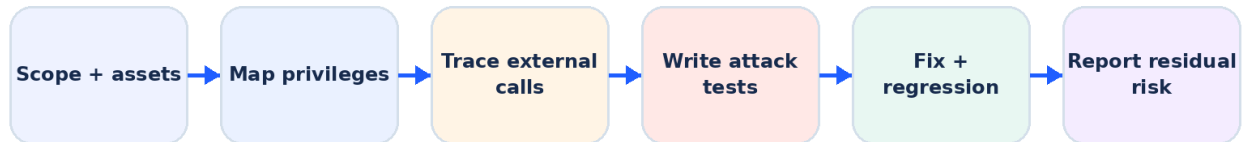
17. Security Test Matrix for Client Contracts

Area	Adversarial test	Expected security property
Access control	Call admin function as random address	Revert; state unchanged
Reentrancy	Receiver callback reenters another entry point	No double-spend / invariant holds
External call failure	Receiver/token deliberately reverts	State remains consistent; failure handled
Signature	Reuse same signed authorization twice	Second use reverts
Signature domain	Attempt same signature in wrong domain/context	Verification fails
Oracle	Supply stale/extreme/manipulated price input	Protocol rejects or remains within bounds
Ordering	Execute adverse transaction before user transaction	Slippage/deadline constraints protect user
Gas / DoS	Test maximum allowed list/batch size	Operation remains callable or safely paginated
Inputs	0, max uint, zero address, wrong decimals	Validated or handled explicitly
Emergency controls	Pause system as authorized role	Sensitive actions stop; recovery path documented

DELIVERABLE QUALITY For an Upwork client, a concise test matrix is valuable because it connects requirements to evidence. It also exposes untested assumptions before deployment.

18. Freelancer Security Review Workflow

Freelancer Security Review Workflow



Security review = reasoning + tests + documentation, not a magic scanner.

A practical review sequence

1. Define scope: contracts, chain, compiler, proxy architecture, external integrations, privileged accounts.
2. List assets and critical actions: withdraw, mint, upgrade, change oracle, change fee, grant roles.
3. Trace all external calls and callbacks. Mark where untrusted code can execute.
4. Trace all authorization paths and role-admin relationships.
5. Write attack hypotheses as tests instead of relying only on visual inspection.
6. Fix the smallest root cause, then run regression + happy-path tests.
7. Report severity, impact, evidence, remediation and remaining assumptions.

SCOPE LANGUAGE A focused code review is not automatically a formal audit. Be precise with clients about scope, methodology, limitations and whether independent audit coverage is still recommended before high-value deployment.

19. How to Write a Security Finding

Field	What to write
Title	Short vulnerability + impact, e.g. "Reentrancy allows repeated withdrawal"
Severity	Critical / High / Medium / Low / Informational according to defined rubric
Affected code	Contract + function + relevant lines/commit
Root cause	The broken assumption or unsafe ordering
Impact	What an adversary can gain or what users can lose
Proof / evidence	Reproducible local test or precise call sequence
Recommendation	Specific design/code change
Regression evidence	Test showing exploit fails and normal behavior still succeeds
Residual risk	Assumptions that remain: admin key, oracle, external dependency, upgrade authority

Example finding summary

HIGH - REENTRANT WITHDRAWAL `withdraw()` transfers ETH before clearing the caller balance. A receiver contract can call `withdraw()` again during the callback and reuse the same balance. Move the balance update before the external call and add `nonReentrant` to the sensitive entry point. Add a regression test with a malicious receiver.

20. Upwork / LinkedIn Client Scenarios

Client request	What you should inspect first	Professional response
"Please secure my staking contract."	Roles, reward accounting, token calls, withdrawal order, pause/admin	Define scope, map invariants, add attack/regression tests
"Anyone can mint our token."	mint visibility, modifiers, role admins, initializer	Identify privilege bug, patch, add unauthorized-caller test
"Our withdrawal was exploited."	External call order, callbacks, cross-function state	Reproduce locally, quantify impact path, fix + regression
"Price can be manipulated."	Oracle source, pool liquidity, freshness, decimals, update cadence	Model adverse price, redesign oracle assumptions
"We use signatures for claims."	Nonce, domain, chain, contract, deadline, signer recovery	Test replay and wrong-domain use
"Can you audit before mainnet?"	Scope, code freeze, proxy/admins, dependencies, test coverage	State what your review covers and recommend independent audit for high-value launch

FREELANCER HABIT Never tell a client "the contract is 100% secure." Report the evidence you produced and the boundaries of that evidence.

21. Interview Questions - Practice Out Loud

1. What is reentrancy?

An external call lets untrusted code call back before the original execution finishes; stale or inconsistent state can then be reused.

2. What is CEI?

Checks-Effects-Interactions: validate first, update internal state second, perform external interactions last.

3. Why is access control critical?

Because privileged functions can move assets or change security-critical configuration even when the rest of the code is correct.

4. Why is tx.origin unsafe for authorization?

An intermediate malicious contract can cause calls while tx.origin still reflects the victim who initiated the transaction.

5. What prevents signature replay?

Unique nonces/identifiers plus domain/context binding and usually deadlines; consume the nonce after successful use.

6. Is a flash loan itself a vulnerability?

No. It supplies temporary capital; the protocol vulnerability is typically an oracle, accounting or economic-design assumption that the capital can exploit.

7. Why are unbounded loops risky?

Their gas cost grows with state size and may eventually make an operation impossible to execute.

8. What should an audit finding include?

Affected code, root cause, impact, evidence/reproduction, remediation, severity and residual assumptions.

22. Day 10 Quiz - 12 Questions

1. A contract sends ETH before decrementing the sender balance. What vulnerability class should you investigate first?
2. What are the three stages of CEI?
3. Why is a nonReentrant modifier not a substitute for understanding state ordering?
4. What is the difference between msg.sender and tx.origin?
5. Name three items that should be bound into an off-chain signature authorization.
6. Why can a DEX spot price be unsafe as a lending oracle?
7. Give two ways transaction-ordering risk can be limited for swaps.
8. Why can paying 10,000 recipients in one transaction be dangerous?
9. What is the purpose of SafeERC20?
10. What should happen to internal state if an external call fails?
11. What is an invariant? Give one example for a vault.
12. Why should a freelancer distinguish a focused security review from a formal audit?

Answer Key

1. Reentrancy.
2. Checks, Effects, Interactions.
3. Reentrancy can be cross-function/cross-contract and guards do not fix every inconsistent-state design.
4. msg.sender = immediate caller; tx.origin = originating transaction account through the call chain.
5. Examples: parameters/action, nonce, verifying contract/domain, chain/domain context, deadline.
6. Temporary trades can manipulate the pool reserves/spot price.
7. Examples: slippage limits, deadlines, commit-reveal/private order flow depending on design.
8. Gas cost grows and a single transaction may become impossible/unreliable.
9. It safely wraps ERC-20 operations across common token return behaviors.
10. Either revert atomically or handle failure while preserving a correct state/invariant.
11. A property that must always hold; e.g. total user claims must not exceed tracked assets.
12. Scope/methodology/assurance differ; clients need accurate expectations and independent review may still be required.

23. Homework & Portfolio Assignment

Homework A - Reentrancy regression

1. Create VulnerableVault and ReentrantAttacker in a local Foundry project.
2. Write a test that proves the attacker receives more than its recorded deposit from a seeded test vault.
3. Implement SecureVault using CEI and ReentrancyGuard.
4. Run the same attacker strategy against SecureVault and prove the exploit no longer succeeds.
5. Add a normal-user regression test to prove withdrawal still works.

Homework B - Access control

- Add ADMIN_ROLE and PAUSER_ROLE to a small contract.
- Test that unauthorized callers cannot grant roles, pause, change treasury or withdraw admin funds.
- Document who holds each role and who is allowed to grant/revoke it.

Homework C - Client-style security note

Write a one-page finding with: title, severity, affected function, root cause, impact, local proof, remediation, regression test and residual risk. Put the report in your GitHub portfolio next to the code and tests.

PORTFOLIO TITLE Smart Contract Security Lab - Reentrancy Detection, Exploit Test & Remediation

24. Day 10 Security Cheat Sheet

Risk	Red flag	Core defense idea
Reentrancy	External call before state finalization	CEI + targeted reentrancy guard + tests
Access control	Sensitive function callable by wrong actor	Least privilege + roles/owner + unauthorized tests
External call failure	Ignored bool/return data	Handle failure; use safe wrappers where appropriate
Front-running / MEV	Outcome depends on transaction order	Slippage/deadline/commit-reveal/private flow as appropriate
Oracle manipulation	Single manipulable spot price	Robust oracle, freshness, TWAP/aggregation/bounds
tx.origin auth	require(tx.origin == owner)	Use explicit caller/role/signature authorization
Signature replay	No nonce/domain/deadline	Nonce + domain/chain/contract binding + expiry
DoS / gas	Unbounded loop / one bad recipient blocks all	Pull payments, bounds, pagination
Input logic	Zero address, wrong units, unchecked assumptions	Validate ranges/units and fuzz edge cases

DAY 10 FORMULA SECURE DESIGN = explicit invariants + least privilege + safe state ordering + distrust of external inputs/calls + adversarial tests + honest documentation.

Prepare for Day 11

Day 11 - Smart Contract Security II: upgradeable contracts, proxy storage, delegatecall risk, governance/admin security, multisigs/timelocks, advanced oracle/MEV thinking, static analysis and deeper audit methodology.

25. Primary References & Further Reading

Solidity - Security Considerations: <https://docs.soliditylang.org/en/latest/security-considerations.html>

Solidity - Latest Documentation: <https://docs.soliditylang.org/>

Ethereum.org - Smart Contract Security: <https://ethereum.org/developers/docs/smart-contracts/security/>

OpenZeppelin Contracts 5.x: <https://docs.openzeppelin.com/contracts/5.x>

OpenZeppelin 5.x Utils - ReentrancyGuard / Pausable: <https://docs.openzeppelin.com/contracts/5.x/api/utils>

OpenZeppelin - Access Control: <https://docs.openzeppelin.com/contracts/5.x/access-control>

Foundry - Forge Testing: <https://getfoundry.sh/forge/testing>

Foundry - expectRevert cheatcode: <https://getfoundry.sh/reference/cheatcodes/expect-revert>

EIP-712 - Typed Structured Data: <https://eips.ethereum.org/EIPS/eip-712>

VERSION NOTE Security tooling and compiler behavior evolve. Before production work, verify current compiler releases, dependency versions, OpenZeppelin APIs and the exact chain/protocol assumptions for the client system.

End of Day 10 - Smart Contract Security I